

## DO BASIC:

### **intros x y.**

O que faz: "Puxa" variáveis quantificadas (forall) ou premissas ( $\rightarrow$ ) que estão no seu objetivo (Goal) para a parte de cima do terminal (Contexto).

### **simpl.**

O que faz: Realiza cálculos e expande definições se a função puder ser executada. (Ex: transforma  $0 + n$  em  $n$ ).

### **reflexivity.**

O que faz: O "xeque-mate". Finaliza a prova se os dois lados do sinal de igual ( $=$ ) forem exatamente a mesma coisa. (Ele faz um simpl automático antes de checar).

### **unfold nome\_da\_funcao.**

O que faz: Desvenda uma função, trocando o nome dela pelo código fonte dela. Útil quando o simpl não funciona direto (Ex: trocar  $\text{oddb } n$  por  $\text{negb } (\text{evenb } n)$ ).

### **rewrite H. (ou rewrite $\rightarrow$ H.)**

O que faz: Se você tem uma igualdade no contexto ( $H: A = B$ ), ele procura o  $A$  no seu objetivo e troca por  $B$ .

### **rewrite $\leftarrow$ H.**

O que faz: O inverso. Procura o lado direito ( $B$ ) e troca pelo esquerdo ( $A$ ).

### **destruct x. (ou destruct x as [alguma coisa].)**

O que faz: Análise de Casos. "Abre" a variável para testar todos os seus valores possíveis.

Quando usar: Quando uma função travou porque tem um if ou match esperando um booleano (true/false) ou um número desconhecido ( $0$  ou  $S\ n$ ).

### **induction n as [| n' IHn'].**

O que faz: O super-poder para listas infinitas (como números). Divide a prova em dois universos:

Caso Base: Substitui  $n$  por  $0$ .

Passo Indutivo: Substitui  $n$  por  $S\ n'$  e te dá a Hipótese de Indução ( $IHn'$ ) como ferramenta.

Quando usar: Quando você quer provar algo sobre uma variável nat arbitrária em uma função recursiva (Fixpoint).

### **replace A with B.**

O que faz: Troca cirurgicamente o pedaço  $A$  pelo pedaço  $B$  no seu objetivo. Em troca, o Coq cria um novo bullet cobrando que você prove que  $A = B$ .

`assert (H : A = B).`

O que faz: Cria um "mini-teorema" no meio da sua prova, dá o nome de H a ele e pede para você provar. Depois de provado, o H fica no seu contexto para você usar com `rewrite`.

## DO TATICS:

### **apply H.**

O que faz: Se o seu objetivo é X, e você tem uma hipótese  $H : X$ , o `apply` resolve a prova em um passo. Se  $H : Y \rightarrow X$  (Y implica X), o `apply H` transforma o seu objetivo X em Y.

### **Diferença pro `rewrite`:**

O `rewrite` troca pedaços (A por B). O `apply` resolve o objetivo inteiro ou aplica regras de causa e consequência.

### **symmetry.**

O que faz: Se o seu objetivo é  $A = B$ , ele inverte para  $B = A$ . Útil quando o `apply` não encaixa por estar do lado "errado".

### **injection H as H1 H2.**

O que faz: Usa a Injetividade. Se  $H : S\ n = S\ m$ , o `injection` cria uma nova hipótese  $H1 : n = m$ . Ele arranca os construtores em comum.

### **discriminate H.**

O que faz: O "Detector de Mentiras". Se você tem uma hipótese no contexto que é uma mentira absurda ( $H : \text{true} = \text{false}$  ou  $H : 0 = S\ n$ ), você digita `discriminate H`. e a prova acaba na hora com sucesso (Princípio da Explosão).

### **tática in H. (Ex: `simpl in H`, `apply L in H`)**

O que faz: Em vez de usar a tática no objetivo, aplica na Hipótese H (Raciocínio para frente).

### **specialize H with (x := valor).**

O que faz: Se a hipótese H começa com um `forall x`, você pode forçar o x a virar um valor específico (Ex: `specialize H with (x := 0)`).

### **generalize dependent m.**

O que faz: O salva-vidas da indução. Pega a variável m que estava no contexto e devolve ela para o Objetivo na forma de um `forall m`.

**unfold nome\_da\_funcao.**

O que faz: Força o Coq a abrir o código fonte de uma função. Útil quando o simpl não funciona.

**destruct (expressao) eqn:E.**

O que faz: Permite fazer análise de casos não apenas em variáveis soltas, mas em contas inteiras (Ex: destruct (n =? 3) eqn:E).

Atenção: O eqn:E é obrigatório aqui, senão o Coq esquece que a conta foi feita e você não consegue provar o resultado!

**DO LOGIC.V:****split.**

O que faz: Quebra um objetivo em dois.

Quando usar: Quando o objetivo for um "E" ( $A \wedge B$ ). Gera um bullet para provar o A e outro para provar o B. Quando o objetivo for um "Se e Somente Se" ( $A \leftrightarrow B$ ).

Gera um bullet provando a "ida" ( $A \rightarrow B$ ) e outro provando a "volta" ( $B \rightarrow A$ ).

**left. / right.**

O que faz: Escolhe um dos lados de um "Ou" para provar.

Quando usar: Quando o objetivo for  $A \vee B$ . Se você digitar left., o objetivo descarta o B e vira apenas A.

**exists valor.**

O que faz: Fornece o "testemunha" (o valor real) que comprova uma existência.

Quando usar: Quando o objetivo for exists x, P x. (Ex: se o objetivo for exists n,  $4 = n + n$ , você digita exists 2., e o objetivo vira  $4 = 2 + 2$ ).

**unfold not.**

O que faz: Revela o que o til ( $\sim$ ) realmente significa no Coq.

Quando usar: Quando você tiver um  $\sim P$  no objetivo ou na hipótese. O Coq vai transformar o  $\sim P$  em  $P \rightarrow \text{False}$  (Se P for verdade, o universo quebra).

**exfalso.**

O que faz: Troca o seu objetivo atual, não importa qual seja, pela palavra False.

Quando usar: Quando você percebe que a sua prova gerou uma mentira cabeluda nas suas hipóteses (Ex: H:  $0 = 1$ ) e o seu objetivo é algo nada a ver (Ex: Goal:  $X = Y$ ). Você dá exfalso, o objetivo vira False, e você usa a sua mentira para provar esse False.

O destruct muda de comportamento dependendo do símbolo que está na sua hipótese H:

**destruct H as [H1 H2]. (Sem barra)**

Quando usar: Se  $H : A \wedge B$  (E). Ele "abre a caixa" e te dá duas hipóteses separadas na mesma tela: H1: A e H2: B.

**destruct H as [H1 | H2]. (Com barra vertical)**

Quando usar: Se  $H : A \vee B$  (OU). Ele divide a sua prova em dois sub-objetivos (bullets). Você terá que provar o seu objetivo assumindo A, e depois provar tudo de novo assumindo B.

**destruct H as [var\_nome H\_nome].**

Quando usar: Se  $H : \text{exists } x, P \ x$  (Existe). Ele retira o valor de dentro do "existe" e dá um nome para ele (ex: x), e dá o nome para a regra que ele satisfaz (ex: H\_nome: P x).

**destruct H. (Em absurdos)**

Quando usar: Se  $H : \text{False}$ . A prova acaba na hora. (Princípio da Explosão).

**rewrite H. (com <->)**

Neste capítulo, se você importar a biblioteca de Setoids (o arquivo já faz isso para você no meio), você pode usar rewrite com hipóteses do tipo  $A \leftrightarrow B$ . Funciona exatamente como um = normal!

Aplicar Teoremas como Funções: `apply (nome_do_teorema arg1 arg2)`.

O que faz: Em vez de dar `apply` teorema. e deixar o Coq adivinhar as variáveis, você já "empurra" o teorema mastigado com os valores que você quer.

Exemplo: Você tem o teorema `plus_O_n` : forall n,  $0 + n = n$ . Se o seu objetivo for  $0 + 5 = 5$ , em vez de reescrever, você pode simplesmente digitar `apply (plus_O_n 5)`. e a prova acaba!

## 2. O "Cheat Sheet" dos Conectivos Lógicos (Essencial)

Para cada operador lógico, você precisa saber como **PROVÁ-LO** (quando está no Objetivo/Goal) e como **USÁ-LO** (quando está nas Hipóteses/Contexto).

| Operador                   | Significado    | Como PROVAR (no Goal)                                                      | Como USAR (na Hipótese H)                                              |
|----------------------------|----------------|----------------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>A /\ B</code>        | E (AND)        | Tática <code>split.</code> (Gera 2 sub-objetivos)                          | <code>destruct H as [HA HB].</code> (Gera 2 hipóteses)                 |
| <code>A \/ B</code>        | OU (OR)        | Tática <code>left.</code> ou <code>right.</code>                           | <code>**destruct H as [HA</code>                                       |
| <code>~ P</code>           | NÃO (NOT)      | <code>unfold not. intros H.</code> (Pois <code>~P é P -&gt; False</code> ) | <code>apply H in...</code> (Se você tiver P, gera <code>False</code> ) |
| <code>False</code>         | Absurdo        | (Impossível provar diretamente)                                            | <code>destruct H.</code> ou tática <code>exfalso.</code>               |
| <code>A &lt;-&gt; B</code> | Se, somente Se | Tática <code>split.</code> (Gera a ida e a volta)                          | <code>rewrite H.</code> (Sim! Funciona como igualdade!)                |
| <code>exists x, P</code>   | Existe         | <code>exists valor.</code> (Você tem que dar o valor de x)                 | <code>destruct H as [x Hx].</code>                                     |

Se for V e quiser usar o destruct tem que ser **destruct H as [ HA | HB ]**.

**Dar destruct em False finaliza a prova (absurdo).**

**Se tivermos False -> False basta aplicar intros [ ].**

## EXEMPLOS COMPLETOS DE MATCH:

```
Definition nandb (b1:bool) (b2:bool) : bool :=
  match b1 with
  | true => match b2 with
    | true => false
    | false => true
  end
  | false => true
end.
```

```
Definition nandb (b1:bool) (b2:bool) : bool :=
  match b1, b2 with
  | true, true => false
  | _, _ => true
end.
```

Prova sinistra:

Theorem zero\_nbeq\_plus\_1 : forall n : nat,  
0 =? (n + 1) = false.

Proof.

intros n. destruct n as [| n'].

- simpl. reflexivity.

- reflexivity.

Qed.

## GUIA DE TRAVAMENTO DA PROVA:

### FASE 1: A Primeira Olhada (O que eu faço no começo da prova?)

#### 1. A Regra do intros:

- O objetivo tem forall x y? Dê intros x y..
- O objetivo tem uma setinha de implicação  $A \rightarrow B$ ? Dê intros H..
-  PERIGO: Se você for fazer indução em n, mas existe um m depois dele (forall n m), não dê intros no m! Ou, se der, use generalize dependent m. antes de chamar a indução. Se você não fizer isso, a sua Hipótese de Indução (IH) vai ficar "engessada" e a prova vai travar lá na frente.

#### 2. O Objetivo (Goal) tem operadores lógicos? ( $\wedge$ , $\vee$ , $\sim$ , $\leftrightarrow$ , exists)

- Se sim, NÃO use indução. Resolva com as regras de lógica da FASE 3.

#### 3. O Objetivo é uma igualdade matemática/lista ( $A = B$ )?

- Tente apenas simpl. reflexivity..
- Se não der certo, você tem que decidir entre destruct e induction (Veja Fase 2).

### FASE 2: O Motor Principal (destruct vs induction)

Como saber qual usar quando a prova trava num cálculo?

- CENÁRIO A: A função trava num if ou =? (Booleano)
  - Sintoma: O simpl não faz nada ou vomita um match na tela.
  - Ação: destruct (n =? m) eqn:E.. Sempre que a dúvida for true ou false, o destruct abre os dois caminhos e o simpl volta a funcionar.

- CENÁRIO B: A variável tem tamanho fixo (Ex: Pares  $X * Y$ )
  - Ação: `destruct p as [x y]..` "Abre a caixa" do par para o Coq enxergar o que tem dentro.
- CENÁRIO C: A variável é infinita (nat ou list) e está "atrapalhando" a função principal (Ex: `n + 0, l ++ []`).
  - Sintoma: A função faz match na esquerda, mas a sua variável desconhecida está na esquerda.
  - Ação: `induction n as [| n' IHn']..` ou `induction l as [| h t IHl]..`
  - No passo indutivo: O seu único objetivo de vida é dar simpl para tentar fazer o lado esquerdo ficar parecido com a IH, dar um rewrite IH, e fechar a prova.

### FASE 3: O "Cheat Sheet" da Lógica (Se tem símbolo lógico, faça isso)

Se o símbolo estiver no Objetivo (Goal), você tem que PROVAR ele. Se o símbolo estiver no Contexto (Hipóteses/H), você tem que USAR ele.

| Símbolo                    | O que é?     | Se está no OBJETIVO (Goal)                                 | Se está na HIPÓTESE (H)                                                                        |
|----------------------------|--------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>A -&gt; B</code>     | Implicação   | <code>intros H.</code>                                     | <code>apply H.</code> (Trás pra frente) <br> <code>apply H in H_fato.</code> (Frente pra trás) |
| <code>A /\ B</code>        | E (AND)      | <code>split.</code> (Abre 2 bullets)                       | <code>destruct H as [H1 H2].</code> (Gera 2 hipóteses)                                         |
| <code>A \/ B</code>        | OU (OR)      | <code>left.</code> ou <code>right.</code> (Escolha 1 lado) | <code>'destruct H as [H1</code>                                                                |
| <code>~ P</code>           | NÃO          | <code>unfold not.</code> <code>intros H.</code>            | <code>apply H in...</code> (Gera explosão / False)                                             |
| <code>False</code>         | Absurdo      | <code>exfalso.</code> (Força o objetivo a ser False)       | <code>destruct H.</code> (Acaba a prova na hora!)                                              |
| <code>A &lt;-&gt; B</code> | Equivalência | <code>split.</code> (Abre Ida e Volta)                     | <code>rewrite H.</code> ou <code>rewrite &lt;- H.</code>                                       |
| <code>exists x</code>      | Existe       | <code>exists valor.</code> (Forneça o valor)               | <code>destruct H as [x Hx].</code> (Abre o pacote)                                             |

### FASE 4: PROTOCOLOS DE EMERGÊNCIA (Deu Branco! O que eu faço?)



Emergência 1: O simpl. vomitou um match gigante na tela.

- O que aconteceu: Você tentou simplificar uma função que tem mais de duas regras (como o `evenb` que pula de 2 em 2), ou que está testando uma variável que o Coq não sabe o que é.
- A Cura: Apague o simpl.. Olhe para a variável que está logo depois do match no texto vomitado. Dê `destruct` nessa variável!



Emergência 2: Os parênteses estão nos lugares errados.

- Sintoma: O objetivo é  $(A ++ B) ++ C = A ++ B ++ C$  ou  $(n + m) + p = n + m +$

p. O reflexivity falha.

- A Cura: Você precisa de Associatividade. Procure (usando Search (`_ ++ _`).) ou crie um Lema Auxiliar provando isso antes do seu teorema principal. Depois use `rewrite app_assoc.` ou `rewrite add_assoc.`

 Emergência 3: O meu objetivo é uma mentira absurda ( $0 = 1$ ).

- Sintoma: Você está num sub-bullet de um `destruct` que não faz sentido existir matematicamente.
- A Cura: Olhe para o seu contexto (Hipóteses). Com certeza tem uma mentira lá também (Ex:  $H : S\ n = 0$  ou  $H : \text{true} = \text{false}$ ). Dê o comando `discriminate H.` e a prova acaba na hora (Princípio da Explosão).

 Emergência 4: Os lados da igualdade começam com as mesmas letras/construtores.

- Sintoma: Hipótese  $H : S\ n = S\ m$  ou  $H : x :: t = y :: l$ .
- A Cura: Use a Injetividade. Dê o comando `injection H as H1..` O Coq vai arrancar as "cascas" ( $S$  ou  $::$ ) e te dar a igualdade do miolo ( $H1 : n = m$  ou  $H1 : x = y$ ).

 Emergência 5: Preciso fazer uma troca cirúrgica e o `rewrite` altera coisas demais.

- Sintoma: Você quer trocar um  $(n + m)$  por  $(m + n)$ , mas o `rewrite add_comm` vira a equação inteira do avesso.
- A Cura: Use `replace (n + m) with (m + n).` O Coq fará a troca só ali, e abrirá um bullet secundário para você provar que a troca era válida (o que você resolve facilmente com o próprio `add_comm`).

 Emergência 6: O nome da função está com uma notação estranha e não consigo provar.

- Sintoma: Exemplo `f (.) f`.
- A Cura: Use `unfold nome_da_funcao.` (ex: `unfold compose.`). Isso retira a "maquiagem" da função e permite que o `simpl` e o `rewrite` voltem a funcionar.

Dica Mental Final para a Prova:

Se você está no meio de uma prova e empacou, faça a si mesmo a pergunta: **"Eu preciso olhar para DENTRO de uma variável ou caixa?"** Se sim, a resposta é `destruct` (em pares, booleanos, ou lógicas) ou `induction` (em listas e naturais).

Boa sorte! Com este mapa mental, você consegue derivar a solução de quase qualquer teorema de Fundamentos Lógicos!



# INDPROP: LÓGICAS E COMANDOS

## 1. A Estrutura de Dados (reg\_exp)

Inductive reg\_exp (T : Type) : Type :=  
| EmptySet  
| EmptyStr  
| Char (t : T)  
| App (r1 r2 : reg\_exp T)  
| Union (r1 r2 : reg\_exp T)  
| Star (r : reg\_exp T).

Isso é apenas a "Sintaxe" (a casca). Não diz nada sobre o que a regex faz. É apenas a forma de escrever.

- EmptySet: A regex nula (representada em livros por \emptyset).
- EmptyStr: A regex vazia (representada em livros por \epsilon).
- **App r1 r2: A concatenação** de duas regex (ex: ab).
- **Union r1 r2: A alternância** (ex: a|b).
- **Star r**: O Fecho de Kleene (ex: a\*).

## 2. A Lógica (A Relação Indutiva exp\_match)

Aqui está a "Semântica" (o coração). A notação  $s \sim re$  significa: "A string 's' casa perfeitamente com a regra 're'".

Lembre-se: s é uma lista de caracteres (ex: [a; b; c]).

Vamos dissecar cada regra de casamento, porque elas serão as ferramentas (apply) e os cenários de indução (induction) dos seus exercícios:

- MEmpty:  $[] \sim \text{EmptyStr}$ 
  - Tradução: A lista vazia é a ÚNICA coisa que casa com a regex EmptyStr.
- MChar:  $[x] \sim \text{Char } x$ 
  - Tradução: Uma lista contendo exatamente um elemento x casa com a regex que pede exatamente a letra x.
- MApp:  $H1: s1 \sim re1 \text{ E } H2: s2 \sim re2 \rightarrow (s1 ++ s2) \sim \text{App } re1 re2$

- Tradução: Se eu colar duas regex (App re1 re2), eu preciso cortar a string no meio. A primeira metade (s1) tem que casar com o re1, e a segunda metade (s2) tem que casar com o re2. O resultado final é a junção das duas metades (s1 ++ s2).
- MUnionL e MUnionR: H1: s1 =~ re1 -> s1 =~ Union re1 re2
  - Tradução: O Union é o nosso "OU" lógico. O Coq é forçado a criar duas regras separadas. A regra L (Left) diz que a string validou o lado esquerdo. A regra R (Right) diz que validou o direito.
- MStar0: [] =~ Star re
  - Tradução: Toda regex com \* (Star) é opcional. Ou seja, ela pode repetir ZERO vezes. Se repetir zero vezes, ela consome a string vazia [].
- MStarApp: H1: s1 =~ re E H2: s2 =~ Star re -> (s1 ++ s2) =~ Star re
  - Tradução: Como funciona o loop infinito do \* em computação funcional? É uma recursão! Para uma string casar com Star re, você tem que "arrancar" o primeiro pedaço dela (s1) e provar que casa com re UMA VEZ. O resto da string inteira (s2) tem que provar que casa com Star re (ou seja, joga o resto no loop de novo). Ele vai fatiando a string em pedaços e consumindo até virar vazia e cair na regra MStar0!

O Segredo de Mestre (Por que o EmptySet sumiu?)

Você notou que não tem nenhuma regra para o EmptySet ali no meio? O EmptySet é uma regex impossível de ser casada por qualquer string do universo. É como se fosse o False lógico.

- O que isso significa na prática: **Se alguma hipótese na sua prova for  $H : s \sim \text{EmptySet}$ , basta você dar o comando inversion H. e a prova vai ACABAR NA HORA** (igual fazíamos com discriminate), porque o Coq vai ver que não existe nenhuma regra criada para casar com isso. O universo explode.

O Guia de Batalha para os Exercícios Seguintes

Nos próximos exercícios, o professor vai te pedir coisas como "Provar que se uma string casa com uma regex, ela deve conter os caracteres da regex".

Como você vai agir:

1. Sempre que ele te der  $H : s \sim re$  e pedir para provar algo, a primeira

- tática é: induction H. (Faça a indução em cima do "casamento").
2. O Coq vai gerar 7 bullets, um para cada construtor acima!
  3. Você terá que resolver 7 mini-provas. As 3 primeiras (MEmpty, MChar, MStar0) saem com simpl. reflexivity.
  4. As provas complexas (MApp, MUnion, MStarApp) vão te dar as Hipóteses de Indução (Ex: "Eu assumo que a metade s1 casou"). Você usará os teoremas antigos do Coq (como In\_app\_iff ou add\_assoc) para juntar os pedaços.

Leia com calma a regra do MStarApp. Se você entender que o loop se divide sempre em "um pedaço único" e "o resto infinito", os exercícios finais ficarão bem claros. Pode mandar o próximo bloco!

**Quanto tem exists em uma resolução COM IMPLICAÇÃO usar: intros [s H] ai ele deixa de existir virando hipotese bonita!**

```
1 goal
T : Type
re : reg_exp T
_____ (1/1)
(exists s : list T, s =~ re) -> re_not_empty re = true
```

**Se o exists ja está em uma hipótese então usar: destruct H1 as [s1 Hm1].**

**ANTES:**

**H1 : exists s : list T, s =~ re1**

**DEPOIS:**

**s1 : list T**

**Hm1 : s1 =~ re1**

## BÔNUS:

### 2. A Diferença entre Indução na Variável vs. Indução na Evidência

Este é o erro número 1 das provas. Você anotou o induction n, mas precisa deixar muito claro quando NÃO usá-lo.

Adicione ao seu resumo:

induction H. (Indução na Evidência / Regra Lógica) O que faz: Em vez de desmontar um número, desmonta a árvore de regras que provou que aquele número é válido.

Quando usar: SEMPRE que você tiver uma hipótese que seja uma propriedade indutiva customizada (Ex:  $H : \text{ev } n$ ,  $H : n \leq m$ ,  $H : s = \sim re$ ).

A Regra de Ouro: Se a função dita as regras (Ex:  $\text{double } n$ ), faça indução na variável (induction n). Se a propriedade lógica dita as regras (Ex:  $\text{ev } n$ ), faça indução na hipótese (induction H). Fazer induction n em números pares ou regex vai travar a sua prova irreversivelmente.

### 3. Inversion vs Destruct (O Descascador de Construtores)

Você anotou o destruct, mas o inversion é a arma mais letal do capítulo de IndProp e Regex.

Adicione ao seu resumo:

inversion H. O que faz: Olha para uma hipótese construída por uma regra e diz: "Para isso ser verdade, o que teve que acontecer antes?". Ele arranca as "cascas" (como S ou ::), extrai as igualdades, e destrói absurdos automaticamente. Quando usar:

1. Quando você tem uma hipótese com construtores iguais dos dois lados (Ex:  $H : S\ n = S\ m$  ou  $H : x :: t = y :: l$ ). Ele vai gerar  $n = m$ .

2. Quando você tem uma hipótese de uma relação lógica aplicada a valores fixos (Ex:  $H : s = \sim \text{EmptySet}$  ou  $H : 0 \leq S\ n$ ). Se a regra for impossível, ele age como o discriminate e acaba a prova. Se for possível, ele te dá as peças que formaram a regra.

**Dica de Ouro:** Sempre que der inversion H., dê o comando subst. logo na linha de baixo. O inversion gera muito lixo no contexto (variáveis como H0, m0). O subst limpa tudo e substitui as letras iguais.